

Java

Introduction

TOPIC 1

Data Types

1.1 Strongly Typed Language

Every variable has a type, every expression has a type, and every type is strictly defined. All assignments are checked for type compatibility at compile time — there are no automatic coercions of conflicting types. Any type mismatch is a compile-time error.

A Scenario from K Motors:

A field storing a vehicle price must be declared as `double`. Attempting to assign a model name string to it is caught by the compiler before the program ever runs.

1.2 The Primitive Types

Java defines eight primitive types of data. The primitive types are also commonly called simple types. They represent single values — not objects. Although Java is otherwise completely object-oriented, the primitive types are not. Making the primitive types into objects would have degraded performance too much. Because Java requires that all programs run the same on all platforms, every primitive type has a strictly defined range. For example, an `int` is always 32 bits, regardless of the particular platform.

A Scenario from K Motors:

Type	Width	Range	K Motors use
byte	8-bit	−128 to 127	Safety star rating (1–5)
short	16-bit	−32,768 to 32,767	Engine RPM at idle
int	32-bit	$\pm 2.1 \times 10^9$	Engine displacement in cc (e.g. 1498)
long	64-bit	$\pm 9.2 \times 10^{18}$	Manufacturer's serial number
float	32-bit	~7 sig. digits	Fuel efficiency (km/l)
double	64-bit	~15 sig. digits	Price (₹)
char	16-bit	0–65,536 Unicode	Fuel type: 'P' 'D' 'E' 'H'
boolean	—	true / false	inStock, isAutomatic

```
int    modelYear    = 2024;
long   serialNo     = 9876543210L;    // manufacturer serial – L suffix required
double price        = 875000.00;
float  fuelEffic     = 17.5f;         // f suffix required for float literals
char   fuelType      = 'E';           // single character
boolean inStock      = true;
```

NOTE



All integer types are signed. Java does not support unsigned integers. Use uppercase `L` for `long` literals — lowercase `l` looks like digit `1`.

1.3 Literals

A literal is a fixed value that is written directly in the source code. Integer literals may be written in decimal (base 10), hexadecimal (base 16, prefix `0x`), octal (base 8, prefix `0`), or binary (base 2, prefix `0b`). You may insert underscores anywhere inside a numeric literal to make it easier to read — the compiler ignores them. Floating-point literals are of type `double` by default; append `f` to make them `float`.

```
int hex    = 0x2A;           // hexadecimal (42)
int bin    = 0b101010;       // binary (42)
int big    = 1_000_000;       // underscores for readability
double sci = 8.75e5;          // 875000.0 – scientific notation
```

1.4 Variables & Scope

A block ({ }) defines a scope. Variables exist only within their scope — they are created on entry and destroyed on exit. Java does not allow a variable in an inner scope to shadow one in an outer scope (unlike C/C++).

```
void applyDiscount(double price) {
    if (price > 500000) {
        double discount = price * 0.05; // only visible in this block
        price -= discount;
    }
    // discount is not accessible here
    System.out.println("Final: ₹" + price);
}
```

1.5 Type Conversion & Casting

If the two types are compatible, Java will perform the conversion automatically. An automatic conversion takes place when the destination type is larger than the source type. This is called a widening conversion. For example, it is always possible to assign an `int` value to a `long` variable. When the destination type is smaller, an explicit cast is required. This kind of conversion is called a narrowing conversion and may result in loss of information. In an expression, `byte`, `short`, and `char` operands are automatically promoted to `int` before the expression is evaluated.

```
double price = 875499.99;
int rounded = (int) price; // 875499 - fractional part lost

byte r1 = 4, r2 = 5;
// byte avg = (r1 + r2) / 2; // ERROR: sum promoted to int
int avg = (r1 + r2) / 2; // correct
```

Operators

2.1 Arithmetic

Java provides the standard arithmetic operators: addition (+), subtraction (-), multiplication (*), division (/), and modulus (%). The modulus operator returns the remainder of a division. When division is applied to two integers, the result is an integer – the fractional part is discarded. The compound assignment operators (+=, -=, *=, /=, %=) combine an arithmetic operation with assignment in a single step. The increment (++) and decrement (--) operators increase or decrease their operand by one.

```
double base = 875000.0;
double gst = base * 0.28; // 28% GST
double onRoad = (base + 45000) * 1.28 - 10000; // accessories + GST - exchange bonus
int kmpl = (int)(totalKm / litres); // integer division truncates
```

In the prefix form (++sold), the operand is incremented before its value is used in the expression. In the postfix form (sold++), the current value is used in the expression first, and then the operand is incremented. This distinction matters only when the operator appears inside a larger expression.

2.2 Bitwise

Bitwise operators are commonly used to pack multiple boolean flags into a single integer field, saving memory and enabling fast membership tests with a single AND operation.

A Scenario from K Motors:

Feature flags are packed into a single int : bit 0 = ABS, bit 1 = airbags, bit 2 = sunroof, bit 3 = reverse camera.

```
int nexon = 0b00001011; // ABS + airbags + reverse cam
int scorpio = 0b00001110; // airbags + sunroof + reverse cam

int common = nexon & scorpio; // 0b00001010 - shared features
int combined = nexon | scorpio; // 0b00001111 - all features
boolean hasSunroof = (nexon & 0b00000100) != 0; // false
nexon |= 0b00000100; // add sunroof flag
```

Java provides three shift operators. The left shift (<<) shifts all bits to the left by the specified number of positions, which is equivalent to multiplying by 2 raised to that power. The signed right shift (>>) shifts bits to the right and fills the vacated high-order bits with the sign bit, preserving the sign of negative numbers. The unsigned right shift (>>>) always fills the high-order bits with zero, regardless of the sign of the value.

2.3 Relational & Logical Operators

The relational operators determine the relationship between two operands. They are: equal to (`=`), not equal to (`≠`), greater than (`>`), less than (`<`), greater than or equal to (`≥`), and less than or equal to (`≤`). Every relational expression produces a `boolean` result — either `true` or `false`.

The logical operators combine boolean values. The short-circuit AND (`&&`) stops evaluating if the left operand is `false`. The short-circuit OR (`||`) stops evaluating if the left operand is `true`. This is both more efficient and safer than the non-short-circuit forms (`&` and `|`), which always evaluate both sides.

```
boolean affordable = price ≤ budget;
boolean isElectric = fuelType == 'E';

// Short-circuit AND: if not in stock, price check is skipped
if (inStock && price ≤ budget) System.out.println("Recommend for test drive");

// Safe division - denominator checked first
if (totalKm ≠ 0 && totalKm / litres > 15) System.out.println("Fuel efficient");
```

NOTE



Prefer `&&` and `||` over `&` and `|` — the short-circuit forms avoid unnecessary evaluation and prevent exceptions like division by zero.

2.4 The Ternary Operator

Java includes a special ternary operator (`? :`) that can replace certain types of if-then-else statements. It has the form: `expression1 ? expression2 : expression3`. If `expression1` is true, the result is `expression2`; otherwise the result is `expression3`. Both `expression2` and `expression3` must return compatible types.

```
String verdict = price ≤ budget ? "Within budget" : "Over budget";
double final_ = returning ? price * 0.97 : price;    // 3% loyalty discount
```

2.5 Operator Precedence

Operators are evaluated in a fixed order of precedence. For example, multiplication is evaluated before addition. When you are not certain of the order, use parentheses to make your intent clear. Parentheses do not reduce the performance of your program in any way.

```
// Without parentheses – GST is applied only to accessories, not the total  
double wrong = base + accessories * gstRate;  
  
// With parentheses – GST is applied to the full amount  
double correct = (base + accessories) * gstRate;
```

Declarations

3.1 Variable Declarations

In Java, all variables must be declared before they can be used. The basic form of a variable declaration specifies the type, followed by the name of the variable. An optional initializer assigns a value at the point of declaration. When an initializer uses an expression that is computed at runtime, this is called dynamic initialization.

```
double price      = 875000.0;           // declared and initialized
double discount;   // declared, not yet initialized
int displacement = 1498;                 // engine displacement in cc
double gst         = price * 0.28;       // dynamic initialization – computed at runtime
```

3.2 Access Modifiers

Access modifiers control visibility of class members.

Modifier	Visible to
private	The declaring class only
(default)	Classes within the same package
protected	Same package + subclasses in any package
public	Everywhere

3.3 static and final

```
class Car {
    static int totalCars = 0;           // shared across all Car instances
    final String vin;                  // cannot be changed after assignment

    Car(String vin) {
        this.vin = vin;
        totalCars++;
    }
}
```

Control Structures

4.1 Selection — if / else-if / else

The `if` statement is Java's conditional branch statement. It routes program execution through different paths based on a boolean condition. The `else` clause is optional and executes when the condition is false. Several conditions can be tested in sequence using an if-else-if ladder. Only the block belonging to the first true condition is executed; all others are skipped.

When the body of an `if`, `else if`, or `else` contains only a single statement, the curly braces are optional. When there is more than one statement, curly braces are required. Both styles are valid Java — many programmers prefer always using braces to avoid mistakes.

Without braces — single statement body

```
double price = 875000.0;
String tier;

if (price < 500000) tier = "Entry";
else if (price < 900000) tier = "Mid-range";
else if (price < 1500000) tier = "Premium";
else tier = "Luxury";

System.out.println(model + " is " + tier);
```

With braces — recommended style

```
double price = 875000.0;
String tier;

if (price < 500000) {
    tier = "Entry";
} else if (price < 900000) {
    tier = "Mid-range";
} else if (price < 1500000) {
    tier = "Premium";
} else {
    tier = "Luxury";
}

System.out.println(model + " is " + tier);
```

4.2 Selection — switch

`switch` can test `int`, `char`, `String`, and `enum` values. Without `break`, execution falls through to the next case.

```
char fuelType = 'D';
switch (fuelType) {
    case 'P': System.out.println("Petrol");    break;
    case 'D': System.out.println("Diesel");    break;
    case 'E': System.out.println("Electric");  break;
    case 'H': System.out.println("Hybrid");    break;
    default: System.out.println("Unknown");
}
```

NOTE



`switch` tests equality only — it cannot evaluate arbitrary boolean expressions like `if` can. For a large number of values, the Java compiler converts the `switch` statement into a `jump table` — a direct lookup of the target address — which executes significantly faster than an equivalent if-else chain.

4.3 Iteration — while

The `while` loop is Java's most fundamental loop statement. It repeats a statement or block while its controlling expression is true. The condition is evaluated at the top of the loop before the body executes. If the condition is false from the beginning, the body of the loop will not execute even once.

```
int i = 0;
while (i < fleet.length) {
    if (fleet[i].inStock) System.out.println(fleet[i].model);
    i++;
}
```

4.4 Iteration — do-while

The `do-while` loop is similar to the `while` loop, except that the condition is evaluated at the bottom of the loop. This means the body of the loop always executes at least once, even if the condition is false from the start. This makes it particularly useful for menu-driven programs where the menu must be displayed before the user can make a choice.

```

char choice;
do {
    System.out.println("1. Search by fuel type");
    System.out.println("2. Search by price range");
    System.out.println("3. Exit");
    choice = (char) System.in.read();
} while (choice < '1' || choice > '3');

```

4.5 Iteration — for

The `for` loop is the most versatile of Java's loop statements. It combines three elements in a single line: initialization, condition, and iteration expression. The initialization is executed once when the loop begins. The condition is evaluated before each iteration; if it is false, the loop ends. The iteration expression is executed after each pass through the body. The enhanced `for` loop, also called the for-each loop, provides a simpler way to cycle through the elements of an array or collection when the index is not required.

```

// Print all cars under ₹10 lakh
for (int i = 0; i < fleet.length; i++) {
    if (fleet[i].price < 1000000)
        System.out.println(fleet[i].model + " ₹" + fleet[i].price);
}

// Enhanced for-each — cleaner when index not needed
for (String model : models) System.out.println(model);

```

4.6 Jump Statements — break & continue

The `break` statement has two uses. First, it terminates a case sequence in a `switch` statement. Second, it forces immediate exit from a loop, bypassing the loop's condition and any remaining code in the body. The `continue` statement is used to skip the rest of the current iteration of a loop and move directly to the next iteration. In a `for` loop, control passes to the update expression first, then to the condition check.

```
// break: stop searching once a match is found
for (int i = 0; i < fleet.length; i++) {
    if (fleet[i].model.equals("Nexon EV")) {
        System.out.println("Found at index " + i);
        break;
    }
}

// continue: skip cars not in stock
for (Car c : fleet) {
    if (!c.inStock) continue;
    System.out.println(c.model + " is available");
}
```

Arrays & Strings

5.1 One-Dimensional Arrays

An array is a group of variables of the same type that are referred to by a common name. Arrays in Java work differently from arrays in C and C++. To create an array, you first declare an array variable of the desired type, then allocate the array using the `new` operator. All array indexes begin at zero. When an array is created, its elements are automatically initialized to zero for numeric types, `false` for `boolean`, and `null` for object references.

```
String[] models = {"Swift", "Nexon", "Scorpio", "Baleno", "Creta"};
double[] prices = {599000, 875000, 1350000, 699000, 1099000};

// Find cheapest
double cheapest = prices[0];
for (int i = 1; i < prices.length; i++)
    if (prices[i] < cheapest) cheapest = prices[i];

System.out.println("Cheapest: ₹" + cheapest);    // ₹599000.0
```

5.2 Multidimensional Arrays

In Java, multidimensional arrays are implemented as arrays of arrays. To declare a multidimensional array variable, specify each additional index using another set of square brackets. When allocating memory, you need only specify the size of the first dimension. The remaining dimensions can be allocated separately, which allows you to create arrays in which the length of each row is different. These are called jagged arrays.

```
// 3 cars x 3 fields: {model, fuelType, variant}
String[][] fleet = {
    {"Swift", "P", "VXi"},
    {"Nexon", "E", "EV Max"},
    {"Scorpio", "D", "Z8"}
};
System.out.println(fleet[1][0] + " " + fleet[1][2]);    // Nexon EV Max
```

5.3 Strings

`String` is not a primitive — it is an object. String variables hold references, not character arrays. Java provides rich string-handling capabilities through the `String` class.

```
String brand = "Maruti Suzuki";
String model = "Swift";
String full = brand + " " + model;    // concatenation
int len = full.length();             // 20
String upper = model.toUpperCase();   // "SWIFT"
boolean isSwift = model.equals("Swift"); // true - always use equals(), not =
```

IMPORTANT



Never compare strings with `=`. Use `.equals()`. The `=` operator compares references (memory addresses), not content.

Input / Output

6.1 Console Output

`System.out` is Java's standard output stream, which is connected to the console. It provides three main methods for displaying output. The `println()` method prints its argument and then moves to a new line. The `print()` method prints its argument without moving to a new line. The `printf()` method prints formatted output using format specifiers, similar to C's `printf`.

```
System.out.println("Welcome to the showroom"); // prints with newline
System.out.print("Price: ₹"); // no newline
System.out.printf("%-15s ₹%,.0f%n", model, price); // formatted output
```

6.2 Console Input with Scanner

`Scanner` is a class in the `java.util` package that reads input from various sources, including the keyboard. To read from the keyboard, pass `System.in` to the `Scanner` constructor. The `Scanner` class provides methods for reading different types of input: `nextDouble()` for decimal numbers, `nextInt()` for integers, `next()` for a single word, and `nextLine()` for a full line of text. When input is complete, call `close()` to release the underlying stream.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter budget (₹): ");
double budget = sc.nextDouble();
System.out.print("Enter fuel type (P/D/E/H): ");
char fuel = sc.next().charAt(0);
sc.close();
```

6.3 Formatted Output with printf

The `printf()` method gives you precise control over how values are displayed. It takes a format string followed by the values to insert. The format string contains *format specifiers* — placeholders that begin with `%` and describe the type and layout of each value. Common specifiers are: `%s` for strings, `%d` for integers, `%f` for floating-point numbers, and `%n` for a new line. You can add width, alignment, and precision between the `%` and the type letter.

```
// printf format specifiers
System.out.printf("Model: %-12s\n", "Nexon EV"); // left-aligned, 12 chars
System.out.printf("Price: ₹%,10.0f\n", 875000.0); // comma-separated, no decimals
System.out.printf("Rating: %d / 5 stars\n", 5); // integer
```

Java Classes

TOPIC 7

Classes & Methods

7.1 Class Fundamentals

A class is a template that defines the state (fields) and behaviour (methods) of objects. An object is an instance of a class — each object has its own copy of the instance variables.

```
class Car {  
    // Fields - instance variables  
    String brand;  
    String model;  
    double price;  
    char    fuelType;  
    boolean inStock;  
  
    // Method - behaviour  
    void printSummary() {  
        System.out.printf("%-15s %-10s ₹%,.0f  [%s]%n",  
            brand, model, price, inStock ? "In Stock" : "Sold Out");  
    }  
}
```

7.2 Declaring and Using Objects

Obtaining an object of a class is a two-step process. First, you declare a variable of the class type. This variable does not define an object — it is simply a variable that can hold a reference to an object. At this point, no memory has been allocated for the object itself. Second, you acquire an actual, physical copy of the object and assign it to that variable using the `new` operator. The `new` operator dynamically allocates memory for an object at runtime and returns a reference to it. This reference is then stored in the variable. Once the object is created, you access its fields and methods using the dot (`.`) operator.

```
// Step 1: declare a reference variable
Car nexon;

// Step 2: allocate the object with new
nexon = new Car();

// Combine both steps (the common form)
Car swift = new Car();

// Assign fields and call a method
swift.brand   = "Maruti Suzuki";
swift.model   = "Swift VXi";
swift.price    = 599000;
swift.fuelType = 'P';
swift.inStock  = true;
swift.printSummary();
```

7.3 Methods — Parameters & Return Values

A method's return type specifies the type of data that the method returns. If a method does not return a value, its return type is `void`. When a method is called, the values of the arguments are copied into the parameters. For primitive types, this means the method receives a copy of the value — changes to the parameter do not affect the original variable. For object references, the method receives a copy of the reference, so the method can modify the object's contents but cannot change which object the caller's variable refers to.

```
class Car {
    double price;

    // Method that returns a value
    double onRoadPrice(double gstRate, double insurance) {
        return price * (1 + gstRate) + insurance;
    }

    // Method that takes and uses parameters
    boolean isAffordable(double budget) {
        return this.onRoadPrice(0.28, 15000) ≤ budget;
    }
}
```

Constructors & Scope Rules

8.1 Constructors

A constructor initializes an object immediately upon creation. It has the same name as the class and no return type — not even `void`. The constructor is automatically called when `new` creates the object. If no constructor is defined, Java supplies a default (no-argument) constructor.

```
class Car {
    String brand;
    String model;
    double price;
    char fuelType;

    // Default constructor - placeholder showroom car
    Car() {
        brand    = "Unknown";
        model    = "Unknown";
        price    = 0.0;
        fuelType = 'P';
    }

    // Parameterized constructor
    Car(String brand, String model, double price, char fuelType) {
        this.brand    = brand;
        this.model    = model;
        this.price    = price;
        this.fuelType = fuelType;
    }
}

// Usage
Car placeholder = new Car();
Car nexon = new Car("Tata", "Nexon EV", 1449000, 'E');
```

8.2 Scope Rules in Classes

Instance variables have class scope — they are visible to all methods in the class. Variables declared inside a method (local variables) are visible only within that method. A local variable with the same name as an instance variable hides the instance variable within that method — use `this` to access the instance variable explicitly.

The `this` Keyword

9.1 Referring to the Current Object

`this` is a reference to the object on which the current method or constructor was invoked. When a constructor parameter has the same name as an instance variable, `this` is used to distinguish the instance variable from the parameter. `this` can also be used inside any instance method to refer to the current object.

```
class Car {
    String brand;
    double price;

    Car(String brand, double price) {
        this.brand = brand;    // this.brand = instance variable
        this.price = price;    // price alone = parameter
    }

    double priceWithGst() {
        return this.price * 1.28;    // this is optional here – no ambiguity
    }
}
```

9.2 Instance Variable Hiding

When a local variable (or parameter) has the same name as an instance variable, the local variable hides the instance variable within that scope. `this.field` always refers to the instance variable, regardless of local variables.

```
class Car {
    double price = 875000;    // instance variable

    void applyDiscount(double price) {    // parameter named 'price' hides instance 'price'
        // price here refers to the parameter
        // this.price refers to the instance variable
        this.price = this.price - price;
    }
}
```



Object-Based vs Object-Oriented Programming

10.1 The Four OOP Principles

Object-oriented programming is organized around four key principles.

Principle	Definition
Encapsulation	The mechanism that binds together data and the methods that operate on that data, and keeps both safe from outside interference and misuse.
Abstraction	The process of hiding implementation details and showing only the functionality to the user. The user knows what a method does, but not how it does it.
Inheritance	The process by which one class acquires the properties and methods of another class. It supports the concept of hierarchical classification.
Polymorphism	A feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.

A Scenario from K Motors:

Encapsulation — price is stored `private` ; outside code uses `getPrice()` only.

Abstraction — `drive()` hides whether the car is combustion or electric.

Inheritance — `ElectricCar` extends `Car` , gaining all `Car` fields and methods.

Polymorphism — `fuelUp()` fills a petrol tank for one car and charges a battery for another, through the same method call.

10.2 Object-Based vs Object-Oriented

An *object-based* language supports encapsulation and objects but does not support inheritance or polymorphism (e.g., early Visual Basic). An *object-oriented* language supports all four pillars. Java is fully object-oriented.

10.3 Encapsulation in Practice

```
class Car {
    private double price;    // hidden – cannot be accessed directly from outside

    public double getPrice() { return price; }

    public void setPrice(double price) {
        if (price < 0) throw new IllegalArgumentException("Price cannot be negative");
        this.price = price;    // validation before storing
    }
}
```

10.4 Composing Classes (HAS-A)

One object can contain another object as a field. This is known as composition, and it models a HAS-A relationship. Composition and inheritance are the two fundamental ways to build complex classes from simpler ones. In composition, the containing class delegates work to the contained object rather than inheriting its behaviour.

```
class Engine {
    String type;            // "Petrol", "Diesel", "Electric"
    double displacement;    // litres

    void start() { System.out.println(type + " engine started"); }
}

class Car {
    String brand;
    Engine engine;          // Car HAS-A Engine – composition

    Car(String brand, Engine engine) {
        this.brand = brand;
        this.engine = engine;
    }

    void drive() { engine.start(); }    // delegate to Engine
}
```

Inheritance & Reusability

11.1 Inheritance Basics

Inheritance allows a new class (subclass) to acquire all the members of an existing class (superclass). Use `extends`. Java supports single inheritance only — a class can have exactly one direct superclass. A subclass is a specialized version of its superclass.

```
class Car {
    String brand;
    String model;
    double price;

    void printSummary() {
        System.out.println(brand + " " + model + " ₹" + price);
    }
}

class ElectricCar extends Car {    // ElectricCar IS-A Car
    double batteryRange;           // km per charge – unique to electric

    void showRange() {
        System.out.println(model + " range: " + batteryRange + " km");
    }
}

// Usage – ElectricCar inherits printSummary() from Car
ElectricCar nexonEv = new ElectricCar();
nexonEv.brand      = "Tata";
nexonEv.model      = "Nexon EV Max";
nexonEv.price      = 1449000;
nexonEv.batteryRange = 437;
nexonEv.printSummary();    // inherited
nexonEv.showRange();       // own method
```

11.2 Using super

`super` has two distinct uses in Java. First, it is used to call a superclass constructor — `super(...)` must always be the first statement in a subclass constructor. Second, it is used to access a member of the superclass that has been hidden by a member of the same name in the subclass. This second use is important when a subclass defines a field or method with the same name as one in the superclass — `super.memberName` explicitly refers to the superclass version.

```

// Use 1: calling superclass constructor
class Car {
    String brand;
    double price;

    Car(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }
}

class ElectricCar extends Car {
    double batteryRange;

    ElectricCar(String brand, double price, double batteryRange) {
        super(brand, price); // must be first - calls Car constructor
        this.batteryRange = batteryRange;
    }
}

// Use 2: accessing a superclass member hidden by the subclass
class Car {
    double price = 875000; // superclass field

    void display() {
        System.out.println("Car price: ₹" + price);
    }
}

class ElectricCar extends Car {
    double price = 1449000; // hides Car.price in this subclass

    void display() {
        System.out.println("Electric price: ₹" + price); // own field
        System.out.println("Base price: ₹" + super.price); // superclass field
        super.display(); // superclass method
    }
}

```

11.3 Multilevel Hierarchy

In Java, a subclass can itself be a superclass for another class. This creates a multilevel hierarchy — a chain of inheritance. A class at any level in the chain inherits all the non-private members of every class above it. Java places no limit on the depth of an inheritance hierarchy.

```

class Car { /* brand, model, price */ }
class ElectricCar extends Car { /* batteryRange */ }
class PremiumElectric extends ElectricCar { /* autopilotLevel */ }
// PremiumElectric inherits from BOTH Car and ElectricCar

```

11.4 Method Overriding

A subclass redefines a method from the superclass with the same signature. The overriding method replaces the superclass version when called on a subclass object.

```
class Car {  
    void fuelUp() { System.out.println("Fill petrol tank"); }  
}  
  
class ElectricCar extends Car {  
    @Override  
    void fuelUp() { System.out.println("Charge battery to 100%"); } // overrides Car  
}  
  
class DieselSUV extends Car {  
    @Override  
    void fuelUp() { System.out.println("Fill diesel tank"); }  
}
```

Method Overloading

12.1 Overloading Methods

Two or more methods in the same class may have the same name provided their parameter lists differ (different number of parameters, or different types). The return type alone is not sufficient to distinguish overloaded methods.

```
class CarCalculator {  
    // Calculate GST from price only (default 28%)  
    double calculateGst(double price) {  
        return price * 0.28;  
    }  
  
    // Calculate GST with custom rate  
    double calculateGst(double price, double rate) {  
        return price * rate;  
    }  
  
    // Calculate GST from price and number of units  
    double calculateGst(double price, int units) {  
        return price * units * 0.28;  
    }  
}
```

12.2 Overloading Constructors

Constructors can be overloaded in the same way that methods can. Providing multiple constructors allows objects to be initialized in different ways, depending on what information is available at the time of creation. Each constructor must have a distinct parameter list. One constructor in a class can call another constructor in the same class by using `this()` as the first statement — this is called constructor chaining and avoids duplicating initialization code.

```
class Car {
    String brand; String model; double price; char fuelType;

    Car() { this("Unknown", "Unknown", 0.0, 'P'); } // delegates to full constructor

    Car(String brand, String model) { this(brand, model, 0.0, 'P'); }

    Car(String brand, String model, double price, char fuelType) {
        this.brand = brand; this.model = model;
        this.price = price; this.fuelType = fuelType;
    }
}

Car placeholder = new Car();
Car partial     = new Car("Tata", "Nexon");
Car full        = new Car("Maruti Suzuki", "Swift", 599000, 'P');
```

Abstract Classes & Dynamic Method Dispatch

13.1 Abstract Classes

An abstract class defines a generalized form that subclasses must complete. It cannot be instantiated directly. Abstract methods have no body — the subclass is required to provide one.

```
abstract class Car {
    String brand;
    double price;

    Car(String brand, double price) {
        this.brand = brand;
        this.price = price;
    }

    // Abstract method – no body, subclass MUST override
    abstract void fuelUp();

    // Concrete method – shared by all subclasses
    void printSummary() {
        System.out.println(brand + " ₹" + price);
    }
}

class PetrolCar extends Car {
    PetrolCar(String brand, double price) { super(brand, price); }

    @Override
    void fuelUp() { System.out.println("Fill petrol tank"); }
}

class ElectricCar extends Car {
    ElectricCar(String brand, double price) { super(brand, price); }

    @Override
    void fuelUp() { System.out.println("Charge battery"); }
}

// Car c = new Car(...); // ERROR – cannot instantiate abstract class
Car c = new ElectricCar("Tata", 1449000); // OK – superclass reference
c.fuelUp(); // calls ElectricCar's version
```

13.2 Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at *run time* rather than at compile time. This is how Java implements runtime polymorphism — one of the most powerful features of object-oriented programming.

Runtime polymorphism means that a single superclass reference variable can refer to objects of different subclass types. When an overridden method is called through that reference, Java looks at the actual type of the object in memory — not the type of the reference variable — and calls the correct version of the method. This decision is made at the moment the program runs, not when it is compiled.

KEY PRINCIPLE



It is the type of the **object** being referred to, not the type of the **reference variable**, that determines which version of an overridden method executes.

```
Car r;                                     // superclass reference variable

r = new PetrolCar("Maruti", 599000);
r.fuelUp();                               // resolved at runtime: calls PetrolCar.fuelUp()

r = new ElectricCar("Tata", 1449000);
r.fuelUp();                               // resolved at runtime: calls ElectricCar.fuelUp()

// The type of r is always Car — but the method called depends on the actual OBJECT
```

13.3 Why Abstract Classes + Dynamic Dispatch?

Together, abstract classes and dynamic method dispatch allow you to write code that works on the general case — the superclass — without knowing the specific subclass that will be used. A method that accepts a `Car` reference can call `fuelUp()` on it and the correct behaviour will be selected at runtime, whether the object is a petrol car, an electric car, or a diesel SUV. When you add a new type of car, you create a new subclass — and all existing code that uses a `Car` reference continues to work without any modification.

A Scenario from K Motors:

The factory floor processes any car through a single `Car` reference, calling `fuelUp()` without knowing whether it's petrol, diesel, or electric. A new fuel type means a new subclass only — no changes to the factory code.

```
Car[] fleet = {
    new PetrolCar("Maruti Swift", 599000),
    new ElectricCar("Tata Nexon EV", 1449000),
    new DieselSUV("Mahindra Scorpio", 1350000)
};

for (Car c : fleet) {
    c.printSummary();    // concrete - same for all
    c.fuelUp();          // abstract - different for each
}
```

Packages & Interfaces

TOPIC 14

Packages & Access Protection

14.1 Defining a Package

A package is a container for classes. It prevents name collisions and controls access. The `package` statement must be the first line in the source file. The file must be stored in a directory matching the package name.

```
// File: com/kmotors/inventory/Car.java
package com.kmotors.inventory;

public class Car {
    public String brand;
    public double price;
    private String internalCode;    // not accessible outside this class
    int lotPosition;               // default – accessible within com.kmotors.inventory
    protected double baseCost;    // accessible to subclasses anywhere
}
```

14.2 Access Protection Summary

Modifier	Same class	Same package	Subclass (diff pkg)	Different package
public	✓	✓	✓	✓
protected	✓	✓	✓	X
(default)	✓	✓	X	X
private	✓	X	X	X

14.3 Importing Packages

The `import` statement brings classes (or entire packages) into visibility so they can be used without full qualification. Import statements appear after the package statement and before class definitions.

```
import com.kmotors.inventory.Car;           // import single class
import com.kmotors.inventory.*;             // import all classes in package
import static java.lang.Math.sqrt;         // static import – use sqrt() directly

// Without import:
com.kmotors.inventory.Car nexon = new com.kmotors.inventory.Car();
// With import:
Car nexon = new Car();
```

Interfaces

15.1 Defining an Interface

An interface specifies *what* a class must do, but not *how*. It is a contract — a pure abstract type with no instance variables and (traditionally) no method bodies. All methods are implicitly `public abstract`; all fields are implicitly `public static final`.

```
// Any car CAN drive itself if it implements this interface ("CAN-DO" relationship)
interface SelfDrivable {
    void accelerate(double targetSpeedKmh);
    void brake(double targetSpeedKmh);
    void steerTo(double latitude, double longitude);
    boolean isNavigating();
}
```

15.2 Implementing Interfaces

A class implements an interface using the `implements` keyword. It must provide a concrete body for every method declared in the interface, or the class itself must be declared abstract.

```
class NexonEV extends Car implements SelfDrivable {
    private boolean navigating = false;

    NexonEV(double price) { super("Tata", price); }

    @Override public void accelerate(double targetSpeedKmh) {
        System.out.println("Nexon EV accelerating to " + targetSpeedKmh + " km/h");
    }
    @Override public void brake(double targetSpeedKmh) {
        System.out.println("Nexon EV braking to " + targetSpeedKmh + " km/h");
    }
    @Override public void steerTo(double lat, double lon) {
        navigating = true;
        System.out.printf("Nexon EV navigating to %.4f, %.4f\n", lat, lon);
    }
    @Override public boolean isNavigating() { return navigating; }
}
```

15.3 Applying Interfaces — the "CAN-DO" relationship

A class can implement multiple interfaces. The power of the interface contract is that the consumer never needs to know the concrete type — only that it fulfils the interface.

A Scenario from K Motors:

The self-driving module works with any car that implements `SelfDrivable` — it doesn't need to know whether it's a *Nexon* or a *Scorpio*. New models plug in without touching the module code.

```
// The startup's autonomous module — works with ANY SelfDrivable car
class AutonomousModule {
    void activateAutopilot(SelfDrivable car, double lat, double lon) {
        System.out.println("Autopilot engaged");
        car.steerTo(lat, lon);
        car.accelerate(60);
    }
}

// Usage
SelfDrivable car = new NexonEV(1449000); // interface reference
AutonomousModule module = new AutonomousModule();
module.activateAutopilot(car, 28.6139, 77.2090); // Delhi coordinates
```

15.4 Interface Variables

Fields declared in an interface are implicitly `public static final`. They can be used as named constants shared across all implementing classes.

```
interface SelfDrivable {
    int MAX_AUTO_SPEED = 120; // implicitly public static final
    void accelerate(double speed);
    // ...
}
```

15.5 Extending Interfaces

An interface can extend one or more other interfaces, inheriting their abstract methods.

```
interface FullyAutonomous extends SelfDrivable {
    void parkAutomatically();
    void summonFromParking();
}
```

Interface Enhancements — Default, Static & Functional

16.1 Default Methods

A default method provides a concrete implementation inside an interface. Classes that implement the interface inherit this implementation without being required to override it. This allows existing interfaces to gain new capabilities without breaking implementing classes.

```
interface SelfDrivable {
    void accelerate(double speed);
    void brake(double speed);
    void steerTo(double lat, double lon);
    boolean isNavigating();

    // Default method – implementing classes get this for free
    default void emergencyStop() {
        System.out.println("Emergency stop – braking to 0");
        brake(0);
    }
}
```

16.2 Static Methods in Interfaces

An interface can contain static methods. They belong to the interface itself (not to any implementing class) and are called using the interface name.

```
interface SelfDrivable {
    // ...
    static boolean isAutonomyLegal(String region) {
        return region.equals("TestZone") || region.equals("Highway");
    }
}

// Called as:
boolean ok = SelfDrivable.isAutonomyLegal("Highway"); // true
```

16.3 Functional Interfaces

A functional interface has exactly one abstract method. It is the target type for lambda expressions. The `@FunctionalInterface` annotation is optional but recommended — it causes a compile-time error if the interface has more than one abstract method.

```

@FunctionalInterface
interface CarFilter {
    boolean test(Car car);    // single abstract method
}

// Using a lambda expression as a CarFilter
CarFilter affordable = car → car.price ≤ 900000;
CarFilter electric   = car → car.fuelType = 'E';
CarFilter kmotorsPick = car → car.price ≤ 1200000 && car.fuelType = 'E';

// Apply a filter to the fleet
for (Car c : fleet) {
    if (kmotorsPick.test(c)) System.out.println("Recommend: " + c.model);
}

```

KEY DISTINCTION



Abstract class: use when subclasses share common state (fields) or a partial implementation. A class can extend only one abstract class.

Interface: use to define a capability contract — what a class *can do*, regardless of where it sits in the inheritance hierarchy. A class can implement any number of interfaces.